L4: A Functional Programming Language for Computational Law

Constitutive Rules as Total Functions, Regulative Contracts as Concurrent Processes

Wong Meng Weng
Centre for Computational Law, Singapore Management University; and Legalese
Singapore
mengwong@legalese.com

The L4 Team
Centre for Computational Law, Singapore Management University
Singapore

ABSTRACT

We present *L4*, a statically typed functional programming language for computational law, together with its semantic foundations and the controlled-natural-language surface syntax that makes it usable by legal domain experts. L4 cleanly separates two readings of legal text that the literature has long distinguished but that prior formalisms tend to conflate. *Constitutive* rules—what counts as what—are encoded as total typed functions in the lineage of Haskell-style functional programming, giving type safety, exhaustive case analysis, and an audit-grade evaluation trace for every decision. *Regulative* rules—who must do what, by when, and what happens otherwise—are encoded in a deontic/temporal layer whose denotation is a set of timed event traces, borrowing heavily from Hvitved's Contract Specification Language (CSL) and, through it, from Hoare's Communicating Sequential Processes and the event calculus. The same source program admits a second, relational reading via the standard functional-to-relational transformation familiar from Mercury and Curry; this lets L4 be evaluated forward as a decision procedure, run backward for abductive planning, and exported as a transition relation to symbolic model checkers, so that the search for legal loopholes becomes the search for reachable "double-bind" states. We describe the language, its implementation (parser, type checker, evaluator, language server, REST/MCP decision service, and multiple compilation backends), and our design rationale for a low-punctuation syntax—indentation in place of parentheses, ditto marks, and phrase-shaped identifiers—intended to let non-programmers read and write law in black and white.

KEYWORDS

computational law, rules as code, controlled natural language, functional programming, deontic logic, process calculi, contract formalisation, legal knowledge representation

1 INTRODUCTION

The corporation, viewed from a sufficient distance, is the sum of its contracts: with shareholders, lenders, customers, vendors, employees, and the state. Yet the legal department—custodian of this most fundamental corporate asset—works in tooling that has barely advanced past the word processor. Where the graphic designer has Adobe's suite and the programmer commands languages, compilers, version control, debuggers, and test frameworks, the lawyer has Microsoft Word. This is a striking asymmetry, because the work is not so different: both the programmer and the drafter translate a tangle of business goals into a precise textual artefact that must anticipate the behaviour of a complex system with many moving parts and adversarial participants.

This paper reports on *L4*, a programming language built to close that gap. L4 is not a document format, a clause library, or a contract-lifecycle database; it is a language, in the sense that PostScript was a language before desktop publishing was a product. Its design rests on three commitments that, taken together, distinguish it from the existing body of work in AI and Law. First, it takes seriously the classical distinction between *constitutive* and *regulative* rules [25, 50], giving each a dedicated and appropriate formal treatment rather than forcing both into a single logic. Second, it grounds its regulative layer in a trace-based, concurrency-aware semantics descended from contract DSLs and process calculi [28, 32], so that multi-party obligations with deadlines and reparations are first-class. Third, it is deliberately engineered as a *controlled natural language* [36]: its concrete syntax minimises punctuation, replaces parentheses with indentation, and admits identifiers that are whole legal phrases, so that a statute and its formalisation can be read side by side and recognised as the same text.

Contributions. We make the following contributions.

- We describe the design of L4 and the rationale for its two-layer architecture: a total typed functional core for constitutive rules (Section 4) and a deontic/temporal layer for regulative rules (Section 5).
- We give the semantic foundations of the regulative layer as a trace-based denotational model, and exhibit the correspondence between L4's surface keywords (`HENCE`, `LEST`, `WITHIN`, `RAND`, `ROR`) and the operators of Hvitved's CSL and Hoare's CSP (Section 5).
- We show how a single L4 program supports two readings—functional and relational—via the function-as-graph transformation used in Mercury and Curry, and how the relational reading underwrites abductive planning queries and model-checking-based loophole detection (Section 6).
- We document the controlled-natural-language design of L4's surface syntax and the empirical motivation for each choice (Section 7).
- We report on the implementation and on early deployments in the public and private sectors (Sections 8 and 11).

2 MOTIVATION: WHY A LANGUAGE, IN THE AGE OF LLMS?

A fair objection greets any new programming language in 2026: why bother? Large language models can already program in every language that matters, and a generation of "vibe coders" has, in effect, stopped writing code by hand, describing intent in prose and letting the model fill in the syntax. If the machine will write the code, why design a new notation for humans to read?

The answer is that law is not like ordinary software, because of what happens when it bites. Most of the time a citizen is content to remain a rational ignoramus about the rules that govern them: the expected cost of understanding a tax schedule or an insurance policy exceeds the expected benefit, so people sign and move on. But the calculus inverts the moment the stakes become concrete and personal—when a person might go to jail, or opens an envelope expecting a bill for $600 and finds one for $6,000. In that moment, ordinary people discover an enormous and entirely rational capacity for detail. They want to understand the rule *themselves*; they are no longer willing to defer to a black box, whether that box is a bureaucrat, a chatbot, or an opaque model whose confident paraphrase may or may not track the actual entitlement. A plausible-sounding generated answer is precisely what they cannot trust, because the disagreement is now adversarial and the counterparty has every incentive to read the words differently.

What such situations demand is a rule that is legible *twice over*: readable by a non-technical human who needs to check the reasoning against their own circumstances, and readable by a computer that can evaluate it, test it, and explain it without hallucinating. This dual-legibility requirement is not new. It is exactly the aspiration that gave us COBOL, whose designers wanted business rules expressed in something a manager could read, and SQL, whose declarative surface was meant to let analysts, not just engineers, ask questions of their data. Both bet that some rules are important enough that people will always insist on having them written down in black and white. L4 inherits that bet and adds one refinement: the black and white should be a *formal* language. Natural-language drafting carries avoidable, self-inflicted defects—lexical ambiguity, scope ambiguity in coordinated clauses, the notorious unreliability of punctuation[1]—that a designed notation can simply forbid. The role of the LLM, in our architecture, is not to replace this notation but to help author and explain it: to draft a first formalisation from source text, and to verbalise a rigorous evaluation trace back to the user in accessible prose. The neural "right brain" handles language; a deterministic "left brain" handles the rule. L4 is the left brain.

3 BACKGROUND AND DESIGN GOALS

Constitutive versus regulative. Searle distinguishes regulative rules, which govern antecedently existing behaviour ("drive on the left"), from constitutive rules, which create the very possibility of that behaviour ("this configuration of pieces *counts as* checkmate") [50]. Hart's primary and secondary rules and Hohfeld's jural relations draw related lines [25, 30]. The distinction is not academic for a formaliser. A constitutive rule—"a person born in the United Kingdom after commencement *is* a British citizen if …"—is a definition: it relates facts to legal classifications and is naturally a *function* from situations to conclusions. A regulative rule—"the buyer *must* pay within thirty days, failing which …"—is about action over time, blame, and remedy, and is naturally a *process*. The two demand different mathematics. A recurring weakness of single-formalism approaches is that one of the two is bent to fit the other: deontic notions are encoded as ordinary predicates and lose their temporal and reparational structure, or definitional computation is shoe-horned into a modal logic where it does not belong. L4's central architectural decision is to give each its own layer.

Isomorphism. Following the tradition of the British Nationality Act as a logic program [51] and the broader rules-as-code movement [41], L4 aims for *isomorphic* encoding: the structure of the formal text should mirror the structure of the source. If a section has paragraphs (a), (b), and (c) joined by "and" and "or", the encoding should exhibit the same conjunction-and-disjunction tree, so that a domain expert can audit the correspondence clause by clause and so that a later amendment to the source maps to a localised change in the code. Isomorphism is simultaneously a correctness discipline and a maintenance strategy; it also drives several surface-syntax choices (Section 7).

Two audiences. L4 must satisfy two readers with incompatible habits. The machine wants an unambiguous grammar with a denotational semantics. The lawyer wants prose, or as close to prose as rigour permits, and has no patience for sigils. The thesis of this paper is that these goals are reconcilable rather than opposed: a language can have a wholly conventional formal semantics and, *at the same time*, a surface syntax tuned as a controlled natural language. Sections 4–6 make the first case; Section 7 makes the second.

4 THE CONSTITUTIVE CORE: A TYPED FUNCTIONAL LANGUAGE

L4's constitutive core is a statically typed, total, higher-order functional language. Types are introduced with `DECLARE`: enumerations and algebraic data types with `IS ONE OF`, and records with `HAS`.

```
DECLARE RiskCategory IS ONE OF
    LowRisk
    MediumRisk
    HighRisk
    Uninsurable

DECLARE Driver HAS
    name           IS A STRING
    age            IS A NUMBER
    accidentCount  IS A NUMBER
    hasTickets     IS A BOOLEAN
```

Functions are introduced by a signature—`GIVEN` naming the inputs and `GIVETH` (synonym `GIVES`) naming the result—followed by a definition with `MEANS`, or, for boolean-valued rules, by `DECIDE . . . IF`. Pattern matching uses `CONSIDER` / `WHEN`, a type-safe, exhaustiveness-checked case analysis.

```
GIVEN driver IS A Driver
GIVETH A RiskCategory
‘assess risk‘ driver MEANS
    CONSIDER driver’s accidentCount
```

[1] The missing-comma cases are a genre of their own; *O'Connor v. Oakhurst Dairy* turned a $5M overtime dispute on the absence of a serial comma.

```
  WHEN 0 THEN IF driver's hasTickets
              THEN MediumRisk
              ELSE LowRisk
  WHEN 1 THEN MediumRisk
  WHEN 2 THEN HighRisk
  OTHERWISE   Uninsurable
```

Three properties of this core matter for law. *Totality and exhaustiveness*: the type checker rejects a `CONSIDER` that fails to cover its cases, so a formalised rule cannot silently omit a category of applicant—the analogue of an unhandled edge case is a compile-time error, not a wrong benefit cheque. *Type safety*: a payout formula cannot accidentally add a date to a currency. *Explainability*: because the core is a pure functional evaluator, every result carries a trace from the top-level question down to the deciding sub-condition, with each step annotated by the rule that justified it. This trace is what the system shows an end user, and what it hands to a language model as ground truth to verbalise, rather than asking the model to reason about the law itself.

An isomorphic example. Consider a fragment of the British Nationality Act, the canonical AI-and-Law benchmark [51]. The statute's nested "or"s and "and"s appear directly in the encoding, and the identifiers are the statutory phrases themselves (backticks let an identifier contain spaces):

```
GIVEN p IS A Person
GIVES A BOOLEAN
DECIDE `is a British citizen` IF
        `is born in the United Kingdom after commencement` p
     OR `is born in a qualifying territory after the appointed
     day` p
  AND       `is a British citizen`        OF `father of` p
        OR  `is a British citizen`        OF `mother of` p
     OR     `is settled in the United Kingdom` OF `father of` p
        OR  `is settled in the United Kingdom` OF `mother of` p
```

Note that the indentation, not parenthesisation, carries the precedence of the boolean tree; we return to this in Section 7. Functions marked `@export` are lifted automatically into a REST endpoint, an OpenAPI schema, and a tool callable by an LLM agent; a single source file thus yields an interactive eligibility wizard with citations back to the rule that decided each branch.

5 THE REGULATIVE LAYER: CONTRACTS AS CONCURRENT PROCESSES

Definitional functions cannot express an obligation. "The buyer must pay" is not true or false of a situation; it is a demand on future conduct that is met or breached as events unfold, possibly triggering a remedy. For this L4 provides a regulative sub-language built from a five-keyword skeleton:

```
PARTY   actor
MUST    action            -- or MAY, SHANT, DO
WITHIN  deadline
HENCE   successContinuation
LEST    failureContinuation
```

`MUST`, `MAY`, and `SHANT` are the deontic modalities (obligation, permission, prohibition); `DO` is an uncommitted action. `WITHIN` attaches a deadline. `HENCE` and `LEST` are the continuations taken when the obligation is, respectively, satisfied or not—and crucially, *what counts as satisfaction depends on the modality*: for `MUST`, performing the action by the deadline takes the `HENCE` branch and letting the deadline lapse takes `LEST`; for `SHANT` the polarity flips, so the prohibition is honoured precisely when the deadline passes with no such action. `LEST` typically chains a *reparation*: a further obligation that repairs the breach, the formal home of contrary-to-duty structure that bedevils Standard Deontic Logic [9, 54].

Trace-based semantics. The denotation of a regulative contract is a function from a timed event trace to an outcome: `FULFILLED`, a `BREACH` carrying blame and a reason, or a *residual* contract describing what remains owed. This is exactly the trace-based model that Hvitved gives for multiparty contracts in his Contract Specification Language [32, 33], itself a descendant of the compositional commercial- and financial-contract DSLs of Andersen, Elsman and Henglein and of Peyton Jones and Eber [3, 44]. A contract *is* the set of event traces that conform to it; conformance checking is trace membership; and the residual is the contract's state after a prefix of events has been consumed—the machine-readable answer to "what is still owed, by whom, and by when." L4's `#TRACE` (Figure 1) is precisely this fold of observed events through the denotation. We borrow a great deal from CSL directly: its catalogue of obligations, permissions, and prohibitions; its deadlines and its reparation (contrary-to-duty) structure; and above all its foundational stance that a contract simply *is* the set of traces that conform to it. To CSL's contract DSL L4 adds the two things it did not set out to provide—a controlled-natural-language surface and an integrated constitutive functional core—so that the predicates a contract branches on are themselves type-checked, executable law rather than opaque atoms. The dual view of a contract as a narrative of timed events that initiate and terminate obligations is also exactly the setting of the event calculus [35], which we take as the logical reading of the same traces (Section 6).

The CSP lineage. Read operationally, the regulative layer is a small process calculus in the tradition of Hoare's Communicating Sequential Processes [28, 29, 48]. The correspondence is direct (Table 1): an obligation `PARTY p MUST a HENCE k` is a guarded event prefix $p.a \rightarrow k$; `HENCE` and `LEST` form a choice resolved by what the environment—the trace—actually does, in the manner of CSP's external choice; `RAND` is parallel composition that synchronises on completion, so a compound breaches if either side does; `ROR` is choice between sub-contracts; `FULFILLED` is successful termination (*SKIP*) and `BREACH` is blame-assigning termination. The one essential addition is *time*: `WITHIN` imposes a deadline, placing L4 with Timed CSP [47] and the timed-automata model that underlies tools such as UPPAAL [2, 5]. Recursion gives recurring obligations—a loan's monthly instalments are a function that re-emits next period's obligation in its `HENCE` branch and a penalised obligation in its `LEST` branch, bottoming out at `FULFILLED`.

Why this matters. Because the regulative layer is a timed concurrent system, the questions a lawyer asks about a contract become questions a verification tool can answer. "Is there a state in which one clause obliges an act that another forbids?" is a reachability query over the product of the parties' processes—a race condition or, in deontic terms, a contradictory conflict. In one of our public-sector deployments (Section 11), exactly such a deontic-temporal double bind was discovered automatically in live regulation: under an unusual interleaving, a party was simultaneously obliged

```
DECLARE Person IS ONE OF Seller, Buyer
DECLARE Action IS ONE OF
    delivery
    payment HAS amount IS A NUMBER

saleContract MEANS
    PARTY  Seller
    MUST   delivery
    WITHIN 3
    HENCE ( PARTY  Buyer
            MUST   payment 100
            WITHIN 7
            HENCE  FULFILLED
            LEST   BREACH BY Buyer BECAUSE "payment not made within 7 days" )
    LEST   BREACH BY Seller BECAUSE "delivery deadline exceeded"

#TRACE saleContract AT 0 WITH
    PARTY Seller DOES delivery     AT 2
    PARTY Buyer  DOES payment 100  AT 5
-- Result: FULFILLED
```

Figure 1: A two-party sale as a regulative contract. `HENCE` threads the seller's delivery into the buyer's payment obligation; `#TRACE` replays a timed event sequence and reports `FULFILLED`, a blame-bearing `BREACH`, or the residual obligation that remains.

Table 1: L4 regulative keywords and their CSL / CSP analogues.

L4	CSL / CSP analogue
`PARTY p MUST a HENCE k`	guarded prefix $p.a \rightarrow k$
`WITHIN t`	timed deadline (Timed CSP clock guard)
`HENCE k`	success continuation
`LEST k'`	failure / timeout cont.; reparation (CTD)
`A RAND B`	synchronising parallel $A \parallel B$
`A ROR B`	choice $A \,\Box\, B$
`FULFILLED`	successful termination (*SKIP*)
`BREACH`	blame-assigning termination
`#TRACE c WITH es`	trace membership $es \in \text{traces}(c)$

and prohibited from the same act. Recasting loophole-finding as state-space search is the bridge to the relational reading we turn to next.

6 TWO READINGS OF ONE PROGRAM: FUNCTIONAL AND RELATIONAL

A function $f : A \rightarrow B$ is, set-theoretically, its graph: the relation $\{(a, f\ a) : a \in A\}$. Functional-logic languages exploit this identity. Mercury annotates predicates with *modes*—which arguments are inputs and which outputs—and *determinism*, then compiles the relation to efficient code in whichever mode is requested [52]; Curry integrates functions and relations through narrowing and residuation, so that a function may be run with unknown inputs and the system searches for those that yield a desired output [4, 24]. L4 inherits this perspective. Its constitutive rules, being total typed functions, compile not only to a forward evaluator but also to a relational form: Horn clauses in the manner of Prolog, or, for the regulative layer, a transition relation over events and fluents in the manner of the event calculus [35]. Logical English [34] demonstrates that this declarative substrate can itself wear a natural-language surface; L4 reaches it from the other direction, starting from typed functional programming and recovering the relational reading by transformation, with Mercury and Curry as the conceptual bridges between the two paradigms.

This second reading is not a curiosity; it is what makes L4 a tool for *analysis* rather than mere execution. Three capabilities follow.

Backward / abductive queries. Run forward, `` `is a British citizen` `` classifies a given person. Run backward, the same rule characterises the situations under which citizenship holds—useful for explanation ("which facts, if changed, would flip this decision?") and, in the regulative layer, for planning: "what is the *latest* I may apply for the loan and still deliver on time?" is an abductive query over the contract's transition relation, of the kind logic programming answers natively but a one-directional evaluator cannot.

Exhaustive scenario generation. Treating a rule as a relation lets a property-based tester enumerate the space of situations and check invariants across all of them, rather than the handful of hand-written examples that legal code typically ships with. An early experiment imported a national benefits calculation—originally a Python rules library with a mere handful of unit tests—into a functional setting and generated tests across the whole domain of scenarios, surfacing miscalculations that the sparse hand-written suite had missed.

Model checking. The transition relation can be handed to symbolic model checkers—NuSMV, SPIN, TLA+, UPPAAL [5, 10, 31, 37]—to search for states that violate a property written at the specification level. Here the letter of the law lives at the object level and the spirit of the law lives as property assertions; a counterexample trace is a loophole, found the way a security researcher finds an exploit. The deontic-temporal conflict of Section 5 is one such counterexample.

L4's implementation already carries the seeds of this multi-target strategy: a query planner reorders relational goals, and an MLIR/WASM backend compiles decision logic to a portable binary (Section 8). The point of principle is that *one* isomorphic source, authored once by a domain expert, feeds all of these analyses; the

```
myDeal MEANS
  Deal WITH
    buyer  IS Party WITH name IS "Alice Avocado"
    seller IS Party WITH name IS "Robert Rich"
    item   IS LIST InventoryItem OF "potato",  12,  Nothing
                                 ^ OF "corn",    120, Nothing
                                 ^ OF "avocado", 2,   Nothing
    price  IS Money OF usd, MoneyAmount OF 100, 00
```

Figure 2: The ditto caret ^ repeats the token above it, letting aligned tabular data read as columns.

functional and relational readings are two views of the same text, not two artefacts to be kept in sync.

7 SURFACE SYNTAX AS CONTROLLED NATURAL LANGUAGE

Everything above concerns semantics, and any number of concrete syntaxes could carry it. The wager of L4 is that the concrete syntax is where adoption is won or lost, and that a formal language can be shaped into a *controlled natural language*—a restricted but rigorous fragment of English [36]—without sacrificing its formal character. The design follows a few principles, each with a specific motivation.

Words, not sigils. L4 prefers familiar words to punctuation and symbols: `AT LEAST` for ≥, `PLUS` for +, `'s` for field access (`driver's age`), `OF` for application. A type signature reads as an English clause—"`GIVEN` a person, `GIVES` a boolean"—rather than as `Person -> Bool`. The cost is verbosity; the benefit is that a reader who has never seen a type signature can still parse the sentence. This directly serves the dual-legibility requirement of Section 2. The genitive clitic `'s` is a small but pleasing overload in this spirit: English writes `'s` both for possession and as the contraction of *is*, so `driver's age` reads naturally as a *has-a* (the field the driver *has*) while the very same clitic glosses as the copula in *is-a* predications—one mark spanning exactly the two relations that object orientation keeps apart as composition and subtyping.

Indentation, not parentheses. Following the offside rule [38], L4 is layout-sensitive, and it puts the layout to specific legal use: the indentation of conjuncts and disjuncts encodes the precedence of the boolean tree, so that the visual shape of the code is the logical shape of the rule. In the British Nationality fragment of Section 4, the alignment of the `OR`s under and beside the `AND` is what disambiguates the structure—no reader need balance a parenthesis, and the indentation can be made to mirror the sub-paragraph numbering of the source, advancing isomorphism.

Ditto marks. Tabular legal and financial text repeats structure down columns; clerks have long abbreviated the repetition with the ditto mark. L4 borrows the convention with a caret, `^`, which stands for the token directly above it, so that aligned tabular data—a schedule of territories, an inventory, a fee table—can be written the way a person fills in a form, with repeated material elided into columns (Figure 2). The point is not keystrokes saved but cognitive load: the eye reads the column as a unit.

Phrase-shaped identifiers and mixfix. Backtick-delimited identifiers may contain spaces, so an identifier can be the exact statutory phrase (`` `is settled in the United Kingdom` ``). Such names may be *mixfix*: arguments interleave with the name, so a predicate reads as a sentence—`` `employee` `works for` `employer` `` rather than `worksFor(employee, employer)`. Together these let the formalisation quote its source, the surface condition for a lawyer to accept that the code says what the law says.

Lineage. L4's surface design inherits from a substantial tradition of controlled and computational language for rules. From Grammatical Framework [46] we take the discipline of treating a surface syntax as a type-theoretical grammar—an abstract syntax with concrete realisations— which points toward multilingual rendering of the same rule. From the wide-coverage parsing tradition of C&C and Boxer [8, 14] we take the goal, run in the opposite direction, of mapping language to logical form, the ingestion problem an LLM now eases. From the enterprise standards SBVR [42] and DMN [43] we take the demonstrated appetite among business analysts—not only programmers—for structured vocabulary, rules, and decision tables; and from CNL efforts such as RuleCNL [20], which compiles a business-rule CNL into SBVR, the specific lesson that a controlled surface can target a formal core without exposing it. L4's distinctive bet relative to these is to pair the CNL surface with an *executable, type-checked* semantics spanning both constitutive functions and regulative contracts, and to make the surface *isomorphic* to legal source text rather than to a business-analyst vocabulary.

We do not claim these choices are free. Verbosity, layout-sensitivity, and phrase identifiers each cost something in concision or in editor support, and we discuss the trade-offs in Section 11. We do claim that they are the right trade-offs for a language whose first reader is a domain expert, and that they are made *at the surface*, leaving the semantics of Sections 4–6 untouched.

8 IMPLEMENTATION AND TOOLING

L4 is implemented in Haskell. The core (jl4-core) comprises a layout-sensitive parser, a Hindley–Milner-style type checker, and an evaluator that produces the explanatory traces of Section 4. Around it sits a tooling stack intended to make a single `.l4` file productive: a Language Server Protocol implementation for in-editor type checking, inline evaluation, and go-to-definition; a REPL with trace and query-planning commands; a decision service exposing `@export`-annotated functions as REST endpoints and as Model Context Protocol tools for LLM agents; a WebAssembly build for in-browser and serverless evaluation; and an MLIR backend that compiles decision logic to a portable `.wasm` binary. The toolchain renders every evaluation as a GraphViz *ladder diagram*, and can transpile a rule set to Markdown for human review or generate a consumer-facing web wizard from the same source. A static analyzer classifies financial contracts against the ACTUS and FIBO standards. The web editor at jl4.legalese.com runs the entire pipeline client-side.

The role of LLMs in this stack is deliberately bounded. Models assist with *ingestion*—drafting a first L4 formalisation from a PDF or a statute (Section 9)—and with *explanation*—verbalising a completed, deterministic evaluation trace for a lay user. They are not in the reasoning loop. The reasoner is the L4 evaluator, whose every step is auditable; the model's natural-language output is guard-railed by,

and checkable against, that trace. This is how L4 proposes to counter hallucination in legal AI: not by trusting the model to reason about the law, but by giving it a rigorous artefact to talk about.

9 INGESTION: THE KNOWLEDGE-ACQUISITION BOTTLENECK REVISITED

The expert systems of the 1980s foundered on the *knowledge-acquisition bottleneck* [19, 27]: encoding a domain demanded a knowledge engineer in slow, costly dialogue with a domain expert, and the brittle result rarely repaid the labour. Legal expert systems were no exception. For all the elegance of the British Nationality Act as a logic program [51], formalising law one statute at a time, through a scarce pairing of programmer and lawyer, never approached the scale of the law itself. L4 attacks the bottleneck from two directions.

By hand, but better. Even hand encoding is easier in a language built as a controlled natural language with isomorphism as a first-class goal (Sections 3 and 7). Because the formal text mirrors the structure and quotes the wording of its source, a domain expert can audit it clause by clause without becoming a programmer. We have found this bridge to bear real weight: non-technical business stakeholders are able to *read* an L4 ruleset and *sign off* on it as a faithful statement of the rules they intend. This is a consequential result, because sign-off is precisely the act by which legal responsibility passes from the engineer who wrote the code to the accountable party who owns the rules—the bridge that a formalism opaque to its domain experts can never build. The knowledge-engineer/expert dyad does not vanish, but the gap across it narrows; and, as our experience suggests, software engineers schooled in requirements analysis are themselves capable wordsmiths who surface the ambiguities a business expert would gloss over. Strong types and exhaustiveness checking catch missing cases at the moment of encoding rather than in production. What hand encoding cannot do is scale: it stays linear in scarce expert hours.

At scale, with the type system as oracle. Large language models change the economics of the first pass. A model can propose a formalisation of a statute or a policy in seconds, shifting the human role from authoring to reviewing. The hazard is the familiar one—a fluent encoding that is subtly wrong—and here L4's static type system and language server are the decisive guardrail. The type checker is a sound, deterministic oracle that rejects ill-typed and non-exhaustive formalisations outright, and the LSP reports each violation inline with a precise location, so a model can be driven in a generate–check–repair loop until the file type-checks, and then until its `#ASSERT` golden tests pass—extending the oracle from well-typedness to behaviour. This is the paper's neuro-symbolic thesis applied to ingestion itself: the LLM proposes and the type system disposes. Strong typing turns open-ended natural-language-to-code generation into a constrained search against a verifier, the discipline that makes type-directed program synthesis tractable, and it lets the system scale ingestion without surrendering correctness to a model's confidence.

Corpus at scale. A type-guarded ingestion loop still needs raw material: machine-readable primary law. The complementary enabler is the decades-long campaign to liberate it. Carl Malamud's Public.Resource.Org and Law.gov, and the litigation establishing that the law and the standards incorporated into it are not locked behind copyright [39], have made vast quantities of primary legal material publicly accessible; projects in that lineage—such as the Legal Data Hunter effort—are now channelling statutes, regulations, and codes into ingestible form at scale. With an open corpus on one side and a type-guarded LLM loop on the other, the isomorphic formalisation of law begins to look tractable at a scale the knowledge-acquisition bottleneck once foreclosed. It does not thereby become automatic: faithfulness to legislative or contractual *intent*, as opposed to structure, remains a matter of human judgement and sign-off (Section 11). But formalisation need no longer proceed one statute at a time, gated by a scarce dyad.

10 APPLICATIONS

Because a single `.l4` source feeds many backends (Section 8), one formalisation supports a spectrum of applications across the legal lifecycle—drafting, analysis, citizen self-help, and operational decisioning—rather than a single product. We sketch four archetypes; the pilots of Section 11 instantiate them.

Citizen-facing expert systems. From the same source that the type checker validates, the toolchain auto-generates an interactive web wizard: a member of the public enters the facts of their situation and is shown their obligations or entitlements, each answer carrying a citation to the deciding rule and the evaluation trace as a plain-language explanation. This is the disruptive wedge of Section 2—the overserved non-consumer who simply wants to know where they stand under a regulation or a policy they signed without reading. Building such an expert system is, in L4, a side effect of having written the rules down at all, not a separate engineering project.

Formal verification and static analysis. The relational reading (Section 6) lets a rule set or a draft contract be handed to model-checking backends—NuSMV, SPIN, TLA+, UPPAAL—for compile-time static analysis: discovery of race conditions and deontic-temporal double binds, of clauses that can never fire or obligations that can never be discharged, and of outright inconsistencies in legislation under drafting or in a contract under negotiation. Loophole hunting becomes exploit hunting. The same machinery turns each party's "dozen scenarios I care about" into a regression suite that runs on every redraft, so that an edit from either side that breaks a previously agreed property is caught immediately rather than litigated later.

Generative drafting back into natural language. The ingestion direction—natural language to formal rule (Section 9)—is the obvious use of an LLM, but the reverse is the more interesting one. Once a rule set is formal and has been *proven consistent*, generative AI can render it back into fluent English: the statutory prose a Parliament can debate and enact, or the clause a counterparty will sign. Here the verified formal artefact is the source of truth and the English is a faithful, regenerable rendering—the inversion the rules-as-code agenda has long sought [41]—rather than the prose being authoritative and the logic an after-the-fact gloss that may silently disagree with it. Because the model is constrained to verbalise a fixed artefact, the prose cannot drift from the logic, and

(via grammar-engineering approaches such as [46]) the same rule can be rendered in multiple natural languages from one source.

Audit-grade enterprise decisioning. At the opposite, high-volume end, the decision service plugs into an enterprise business-rules engine for operational decisions—approving or denying loan applications, adjudicating insurance claims, determining benefit eligibility. The differentiator is not throughput but accountability: because every decision carries a deterministic trace from the top-level question down to the deciding condition, the outcome is explainable and auditable *by construction*. This directly answers the transparency expectations that the GDPR's provisions on automated decisions gesture at [17]—and whose precise extent is contested [55]—and that the EU AI Act makes concrete for high-risk systems [18]. The applicant is entitled to a reason, and L4 produces the reason as a first-class artefact: a citable derivation, not the post-hoc rationalisation of a model that, in effect, had a gut feeling about them.

These archetypes are not four products but four views of one source; the platform ambition (Section 13) is precisely that an ecosystem can build all of them, and others not yet imagined, atop rules authored once.

11 EXPERIENCE AND OPEN QUESTIONS

L4 has been exercised in pilots across the public and private sectors. With a government agency we encoded a piece of secondary legislation for regulatory compliance and generated a web wizard that lets members of the public enter their circumstances and see their obligations, each answer carrying citations to the source rules; the formal-verification pass uncovered the deontic-temporal conflict described in Section 5. In the private sector, formalising a major insurer's policy revealed an ambiguity in a payout formula with material financial exposure. A legislative drafting office has explored L4 for future drafting as a rules-as-code exercise of the kind the OECD has urged [41]. A payments fintech used L4 to ingest commercial agreements dense with fee tables and serve them through an SQL-like API for integration with operational systems.

These engagements also exposed the language's rough edges. Layout sensitivity, prized for isomorphism, can frustrate authors pasting from word processors that mangle whitespace. Phrase-shaped identifiers strain editor tooling built for token-shaped names. The regulative layer's defaults for omitted `HENCE` and `LEST` clauses, while convenient, can surprise authors who have not internalised the modality-dependent polarity of "success." Each is a live design question. More fundamentally, the gap between an isomorphic formalisation and a *faithful* one—between matching the structure of the text and matching the intent behind it—is not something the language can close on its own; it remains, as it should, a matter for human legal judgement, now better supported by tools.

12 RELATED WORK

L4 sits at the confluence of several lines of research; its contribution is their combination rather than any single strand.

Contract DSLs and process calculi. The financial-contract pearl of Peyton Jones and Eber [44] and the commercial-contract algebra of Andersen et al. [3] established the compositional, combinator style that Hvitved's CSL [32, 33] generalised to multiparty contracts with a trace semantics and blame assignment; L4's regulative layer is a direct descendant, adding a controlled-natural-language surface and integration with a constitutive functional core. Stipula [13] likewise models legal contracts as a process calculus with state and time, reinforcing the contracts-as-processes view from an automata angle.

Logic programming of law. The British Nationality Act project [51] is the foundational demonstration of isomorphic legislative encoding; PROLEG [49] and the broader argumentation tradition [6, 45] continue it. Logical English [34] shares L4's controlled-natural-language ambition while remaining within logic programming; L4 differs in starting from typed functional programming and reaching the relational reading by transformation (Section 6) rather than as its native substrate.

Deontic and defeasible reasoning. Defeasible deontic logic and the Regorous line of work [22, 23] model normative positions, permissions, and contrary-to-duty structure with great care. L4's reparation-via-`LEST` is a deliberately operational, less expressive treatment of the same contrary-to-duty phenomena [9], chosen for executability and for its fit with the trace semantics.

Languages for law and smart contracts. Catala [40] is the closest contemporary in spirit: a language for legislation, grounded in default logic, with a strong correctness story. L4 differs in its explicit constitutive/regulative split, its concurrency-aware contract semantics, and its CNL surface. DAML [15] brings a Haskell-derived functional language to ledger contracts but targets execution on distributed ledgers rather than isomorphic formalisation of existing law. The symbolic-drafting tradition runs back to Allen [1] and, earlier still, to Coode's nineteenth-century anatomy of legislative sentences [12].

13 CONCLUSION AND FUTURE WORK

We have presented L4, a functional programming language for computational law that separates constitutive rules, treated as total typed functions, from regulative rules, treated as timed concurrent processes with a trace-based denotation in the lineage of CSL and CSP. We have shown that a single isomorphic source admits both a functional reading, for evaluation and explanation, and a relational reading, for abductive planning and model-checking-based loophole detection; and we have argued that the surface syntax can be engineered as a controlled natural language—low on punctuation, layout-structured, phrase-named—so that the same text is legible to the lawyer and to the machine. The motivation is not technical novelty for its own sake but the durable demand, sharpened in the age of generative AI, for rules a person can check for themselves precisely when the law begins to bite.

Future work runs along three axes: deepening the formal correspondence with CSL and Timed CSP to a full proof of adequacy for the regulative semantics; maturing the relational backends so that conflict detection and abductive planning are push-button for non-experts; and a controlled study of authorability, measuring whether L4's CNL design in fact lowers the barrier for legally trained, non-programmer authors. The broader aim is a platform: a language at the bottom of the stack, in the manner of PostScript, on which an ecosystem of legal applications—wizards, checkers, planners,

integrations—can be built atop rules written once, in black and white, and read by everyone.

ACKNOWLEDGMENTS

The authors thank colleagues at the Centre for Computational Law and the legal and engineering collaborators on the pilots described herein.

A EXTENDING L4 TO PROLEG: A BURDEN-OF-PROOF MONAD

This appendix sketches an extension of L4 toward *PROLEG* [49], Satoh's logic-programming rendering of the "presupposed ultimate fact theory" of the Japanese Civil Code, and the theory of the bridge we are building between the two.[2] PROLEG is an attractive target for three reasons aligned with the body of this paper: it is a large, real corpus of formalised law (some 2,500 rules, validated against decades of bar examinations), so it stresses ingestion at scale (Section 9); it is purely *constitutive* and so exercises L4's functional core and its relational reading (Section 6); and its native treatment of *burden of proof* forces a distinction the body of the paper left implicit—a third jural role, neither constitutive definition nor regulative obligation.

A.1 Canonical PROLEG

Canonical PROLEG is ordinary Prolog augmented with a rule arrow `<=` and a defeasibility operator `exception(H,E)`: rule `H` is defeated when its defeater `E` is provable, and exceptions nest. Layered on top is a litigation vocabulary—`allege`, `provide_evidence`, `admission`, `plausible`—that encodes who must establish each ultimate fact and to what standard. Figure 3 shows the core of the canonical lease fixture: a landlord's cancellation claim, an approval-of-sublease defence, and an abuse-of-confidence exception that defeats the tenant's *non*-abuse defence—an exception of an exception. The fixture is drawn from genuine Japanese law. Article 612 of the Civil Code provides that a lessee may not assign or sublease without the lessor's consent, and that upon an unauthorised sublease the lessor may cancel the contract; the Supreme Court's judgement of 27 January 1966 (Minshū 20-1-136) qualified this with the breach-of-trust, or *abuse of confidence*, doctrine, under which cancellation is barred where the sublease does not in fact betray the parties' relationship of trust. That qualification is the exception of an exception the fixture encodes; the original statutory text is reproduced in the companion Japanese translation of this paper.

A.2 Two modes of translation

We translate in two modes that are, importantly, not two compilers.

Mode B (decision-only). Erase the burden layer and read each rule purely constitutively: `H <= B` becomes `DECIDE H IF B`, multiple rules for one head become a disjunction, and each `exception(H,E)` becomes a conjoined `AND NOT E`. Figure 4 is the Mode-B image of Figure 3. The translation is the classical reading of stratified negation-as-failure [11, 21], and is sound exactly when the signed dependency graph is stratified—a condition the transpiler must check, not assume.

Mode A (judgement-faithful). Retain the burden of proof and reify it as an L4 library. The key observation is that Mode A is Mode B *plus* typing each fact as a burden-bearing value and adding a `resolve` pass; `resolve` is exactly the Mode-A-to-Mode-B desugaring. One compiler, two fidelities.

A.3 A clause-by-clause walkthrough

A translation earns the word *isomorphic* only if a reader can lay the two texts side by side and check them clause by clause. We walk the lease case of Figure 3 from rulebase to factbase to judgement, to make the correspondence explicit.

Rules. The mapping is local and structural. Two PROLEG rules that share a head—`contract_end <= cancellation_due_to_sublease` and `contract_end <= expiration_. . .`—become one `DECIDE` whose body is their disjunction. A rule together with the `exception/2` clauses that name its defeaters—which PROLEG writes as *separate* top-level clauses—gathers into a single `DECIDE` whose body conjoins the rule's premises with one `AND NOT` per defeater. This per-head gathering is the sole normalisation the translation performs, and it is what makes the L4 rule readable as a self-contained statement of when its head holds. Thus the six-premise `cancellation_due_to_sublease` rule and its two exceptions collapse into the single declaration of Figure 4, and the chain `nonabuse <= fact_of_nonabuse` / `exception(nonabuse, abuse)` / `abuse <= fact_of_abuse` becomes the two declarations that

```
cancellation_due_to_sublease <=
    agreement_of_lease_contract, handover_to_lessee,
    agreement_of_sublease_contract, handover_to_sublessee,
    using_leased_thing, manifestation_cancellation.
exception(cancellation_due_to_sublease, get_approval_of_sublease).
exception(cancellation_due_to_sublease, nonabuse_of_confidence).
nonabuse_of_confidence <= fact_of_nonabuse_of_confidence.
exception(nonabuse_of_confidence, abuse_of_confidence).
abuse_of_confidence    <= fact_of_abuse_of_confidence.
```

Figure 3: Canonical PROLEG: the lease-cancellation rulebase [49], with two defeaters and an exception-of-exception.

```
DECIDE ‘cancellation due to sublease‘ IF
        ‘agreement of lease contract‘
    AND ‘handover to lessee‘
    AND ‘agreement of sublease contract‘
    AND ‘handover to sublessee‘
    AND ‘using leased thing‘
    AND ‘manifestation cancellation‘
    AND NOT ‘get approval of sublease‘    -- exception
    AND NOT ‘nonabuse of confidence‘      -- exception

DECIDE ‘nonabuse of confidence‘ IF
        ‘fact of nonabuse of confidence‘
    AND NOT ‘abuse of confidence‘         -- exception-of-exception
```

Figure 4: Mode-B L4: defeasibility becomes `AND NOT`; the exception-of-exception falls out structurally. The output is plain constitutive L4, validated by `jl4-cli`.

[2] The two-way PROLEG↔L4 parser and transpiler are under active development in a parallel effort; what follows is a design-and-theory note, not a report on a finished system. The front end already round-trips canonical PROLEG (`parse . print = id`) on the fixtures discussed below, and the burden-of-proof library of §A.5 type-checks and reproduces the reference judgement; the Mode-B emitter is in progress.

```
admission(agreement_of_lease_contract, defendant). % +5 constitutive
allege(approval_of_sublease, defendant).
provide_evidence(approval_of_sublease, defendant). % but NOT
     plausible
allege(fact_of_nonabuse_of_confidence, defendant).
plausible(fact_of_nonabuse_of_confidence).         % => established
allege(fact_of_abuse_of_confidence, plaintiff).
plausible(fact_of_abuse_of_confidence).            % => established
```

Figure 5: The lease factbase (excerpt). `admission` by the opponent and `plausible` both establish a fact; an `alleged` fact never made `plausible` stays unestablished—and its bearer loses on it.

close it. Rule for rule, the L4 file has the same shape as the PROLEG one.

Facts. The factbase (Figure 5) is where the burden vocabulary does its work. A fact counts as *established* when its proponent meets the standard—`plausible(F)` holds—or when the opposing party concedes it by `admission`. Reading the lease factbase through that rule: the six constitutive facts of the cancellation claim are `admission`-ed by the defendant, and so are established; the two approval facts are `alleged` and evidenced by the defendant but never made `plausible`, and so are *un*established; while `fact_of_nonabuse` and `fact_of_abuse` are each `plausible`, and so established. In Mode B these resolve to the booleans the `DECIDE`s consume; in Mode A they are the `established`/`unestablished` injections of Figure 9, each carrying its bearer into the ledger.

Judgement. Evaluating `contract_end` now descends the rules deterministically:

- `contract_end` reduces to `cancellation OR expiration`; the expiration branch has no established facts and fails, so everything rests on cancellation.
- `cancellation` requires its six constitutive facts (all established by admission), `AND NOT get_approval`, `AND NOT nonabuse`.
- `get_approval` needs both approval facts, which are unestablished, so it fails—and `NOT get_approval` holds. The defendant's first defence is out.
- `nonabuse` needs `fact_of_nonabuse` (established) `AND NOT abuse`. But `abuse` needs `fact_of_abuse`, which *is* established, so `abuse` holds, `NOT abuse` fails, and `nonabuse` fails—so `NOT nonabuse` holds. This is the exception of an exception: the plaintiff's abuse of confidence defeats the defendant's non-abuse defence.
- With both defeaters out, `cancellation` holds, `contract_end` holds, and the landlord prevails.

This descent *is* the L4 evaluation trace—the ladder diagram of Section 8—and exactly what the `#ASSERT`s of Figure 9 check. Because every step cites a rule that exists one-for-one in the source, the trace doubles as an audit of the PROLEG program: the explanation a reviewer reads is a literal walk over the statute's own structure. The burden ledger then adds the dimension Mode B discards—`fact_of_abuse` on the plaintiff, `fact_of_nonabuse` on the defendant—recovering, for free, the question of who would have lost had each fact gone unproven.

```
rescission_by_minor_buyer(Buyer,Seller,CID,Tc,Tr) <=
    minor(Buyer),
    manifestation(rescission(CID),Buyer,Seller,Tr),
    before_the_day(Tc,Tr).
exception(manifestation(A,Mfr,Mfe,Ta),
          manifestation_by_duress(Thr,Mfr,Mfe,A,Ta,Td,Tr)).
```

L4 (Mode B, types reconstructed):

```
GIVEN buyer IS A Person, seller IS A Person,
      contract IS A ContractId,
      tContract IS A Date, tRescission IS A Date
GIVETH A BOOLEAN
DECIDE ‘rescission by minor buyer‘
       buyer seller contract tContract tRescission IF
        ‘minor‘ buyer
    AND ‘manifestation‘ (rescission contract) buyer seller
      tRescission
    AND tContract ‘before‘ tRescission

DECIDE ‘manifestation‘ action mfr mfe tAction IF
       ‘manifestation fact‘ action mfr mfe tAction
    AND NOT ‘manifestation by duress‘ ...   -- duress defeats it
```

Figure 6: First-order PROLEG and its Mode-B image. Positional untyped arguments are reconstructed into named, typed parameters; the exception-of-exception (duress defeats the minor's rescission, so the sale stands) survives the translation.

A.4 First-order rules and type reconstruction

The lease fixture is propositional, but most of PROLEG is first-order: predicates carry positional, untyped arguments. Figure 6 shows a fragment of a second fixture—a minor's rescission of a sale, defeated by duress—in which the arguments are buyers, sellers, contracts, and dates. Translating it to L4, whose functions are typed and whose fields are named, requires *type reconstruction*: inferring from usage that the first argument of `rescission_by_minor_buyer` is the same sort as the `Buyer` flowing into `minor` and `manifestation`, then giving that position a name and a type. This is Hindley–Milner-style inference run across the predicate signatures, and it is where untyped PROLEG and typed L4 most visibly diverge—the temporal arguments, for instance, become L4 `Date` values with a real ordering (Section 5), not opaque atoms.

Reconstruction is not merely a convenience: it is a *linter*. The published slides for this very example contain typos—a defeater spelled `minifestation_by_duress`, variables `Maniester` and `Mnifestee`—that untyped Prolog accepts silently, since a misspelt functor is simply a fresh predicate that never unifies (a defeater that can never fire, and so a bug that quietly changes the judgement). L4's name resolver and type checker reject exactly these as unbound or ill-typed. The type-as-oracle ingestion loop of Section 9 therefore pays a second dividend here: round-tripping PROLEG's corpus through L4 makes the corpus audit itself, surfacing the silent drafting errors that a large body of untyped rules inevitably accretes.

A.5 The burden-of-proof monad

The recurring move in all three traditions the paper touches—deontic obligation, residual control, and burden of proof—is to index a normative position by a responsible party, (*position*, *party*).

```haskell
data Subject    = Plaintiff | Defendant
data Obligation = Obligation { onWhom :: Subject, what :: Text }

-- truth (Maybe) over a ledger of burdens (Writer); MaybeT is
-- outside, so Provable a  =  ([Obligation], Maybe a)
type Provable = MaybeT (Writer [Obligation])

prove      :: Subject -> Text -> Maybe a -> Provable a -- a borne
     fact
notProven  :: Provable a -> Provable ()  -- negation as failure
flipBurden :: Provable a -> Provable a   -- involution = opposite(P)
absent     :: Provable a -> Provable ()  -- defeater, opposing party
resolve    :: Provable a -> Bool         -- close the world
```

Figure 7: The burden monad `L4.Proleg.Burden`. `flipBurden` is PROLEG's `opposite(P)` localised to the exception boundary; `absent = notProven . flipBurden` is a defeater the opposing party must establish.

By Hohfeld [30] these are three *distinct* positions: the obligation-bearer who is blamed on breach (a duty, the agentive subject of CSL and of L4's `PARTY ... MUST`, Section 5); the residual-control holder who decides in the gap (a power); and the bearer of the risk of non-persuasion who *loses* if a fact is unproven (a liability). The construct here formalises *only* the third, and the governing transpiler rule is that a PROLEG party is a burden-role and must *never* be synthesised into an L4 obligation.

A proposition's evidential state has two independent dimensions: its *truth* (established, refuted, or undecided—naturally `Maybe`, with `Nothing` the undecided case) and its *burden* (who must establish it). "A value carrying its evidential subject" is the coreader / environment comonad $(Subject, a)$ [53]; it becomes a *monad* precisely when the subject carries a monoid—that is, once one decides how two burdens combine. There is no sensible *plaintiff* $\diamond$ *defendant*, so we do not merge subjects but *accumulate* them; the free monoid over elementary burdens is an *obligation ledger*, and the comonad-with-monoidal-output is then a `Writer` monad [56]. Stacked over the truth dimension in the order that keeps the ledger even on a failed proof—so that blame survives failure—this is `MaybeT (Writer [Obligation])` (Figure 7). The ledger emitted as the `Writer` output *is* the responsibility map, generated for free by composition; and because conjunction is accumulating rather than short-circuiting, that map is structural (it reflects the rule, not the run).

A.6 The construct in L4 itself

The construct is not only Haskell theory; it is realised in L4 itself. Figure 8 excerpts `burden.l4`. Because L4 has no `Monad` typeclass, the monad lives in the evaluator, and the same algebra surfaces at the source level as a record carrying the truth dimension (`MAYBE BOOLEAN`, with `NOTHING` the undecided case) and an accumulating obligation ledger, together with explicit combinators. `notProven` is negation as failure; `flipBurden` relabels a sub-derivation's obligations to the opposite party; `resolve` closes the world, so `NOTHING` becomes `FALSE`—the bearer loses. The genitive accessor `p's truth`—the very `'s` overload of Section 7—reads the record field.

The point of writing it in L4 rather than on paper is that the worked judgement becomes executable. Figure 9 encodes the lease case of Figure 3 directly and states the expected outcome as `#ASSERT`s

```
DECLARE Subject IS ONE OF Plaintiff, Defendant

DECLARE Obligation HAS
    onWhom IS A Subject
    what   IS A STRING

-- Haskell: Provable = MaybeT (Writer [Obligation]).  L4 has no
     Monad
-- typeclass, so the algebra surfaces as a record: a truth dimension
-- (NOTHING = undecided) plus an accumulating obligation ledger.
DECLARE Provable HAS
    truth       IS A MAYBE BOOLEAN
    obligations IS A LIST OF Obligation

GIVEN p IS A Subject, claim IS A STRING, mx IS A MAYBE BOOLEAN
GIVETH A Provable
prove p claim mx MEANS
    Provable WITH
        truth       IS mx
        obligations IS LIST (Obligation WITH onWhom IS p, what IS
     claim)

-- negation as failure: holds iff its argument is not established
GIVEN p IS A Provable
GIVETH A Provable
notProven p MEANS
    Provable WITH
        truth       IS (IF isTrue (p's truth) THEN NOTHING ELSE
     JUST TRUE)
        obligations IS p's obligations

-- relabel obligations to the opposite party (an involution)
GIVEN p IS A Provable
GIVETH A Provable
flipBurden p MEANS
    Provable WITH
        truth       IS p's truth
        obligations IS flipAll (p's obligations)

-- close the world: established => TRUE, else the burden default
resolve p MEANS isTrue (p's truth)
```

Figure 8: Excerpt of `burden.l4`: the burden construct written in L4. The record `Provable` is the L4 image of `MaybeT (Writer [Obligation])`; the combinators are the source-level monad operations.

that `jl4-cli` checks: `contractEnd` and `cancellation` hold, while the approval defence and the non-abuse defence do not—the latter defeated by the plaintiff's abuse of confidence, an exception of an exception. The closing `#EVAL` prints the responsibility ledger; `flipBurden` dualises it, re-attributing each fact to the party who bears its converse.

A.7 The presumption is the monoidal identity

Promoting the structure from a semigroup to a monoid forces a choice a semigroup never makes: the identity element—the value of a proposition *before any evidence is combined in*. Legally, this unit is the *presumption*, and fixing it is a constitutional act. The carrier `Bool` admits two monoids, De Morgan duals (Table 2): under $(\vee, \mathit{False})$ a charge is not established until a sufficient ground flips it true—the presumption of innocence; under $(\wedge, \mathit{True})$ a claim stands until disproof. Same elements, same operation, *different unit*—*different morality*; `flipBurden` De Morgan-dualises the one into the other, mirroring $\neg\neg = \mathrm{id}$. A *rebuttable* presumption is the identity

```
-- defendant's first defence: alleged but NOT plausible =>
     unestablished
getApproval MEANS
  conj (LIST (unestablished Defendant "approval_of_sublease"),
             (unestablished Defendant
     "approval_before_cancellation"))

-- plaintiff's exception-of-exception
abuse MEANS conj (LIST (established Plaintiff
     "fact_of_abuse_of_confidence"))

-- defendant's second defence, itself defeated by the plaintiff's
     abuse
nonabuse MEANS
  conj (LIST (established Defendant
     "fact_of_nonabuse_of_confidence"),
             (notProven abuse))

cancellation MEANS
  conj (LIST (established Plaintiff "agreement_of_lease_contract"),
             -- ... the other five constitutive facts ...
             (established Plaintiff "manifestation_cancellation"),
             (notProven getApproval),
             (notProven nonabuse))

contractEnd MEANS disj (LIST cancellation)

#ASSERT      resolve contractEnd
#ASSERT      resolve cancellation
#ASSERT NOT (resolve getApproval)
#ASSERT NOT (resolve nonabuse)

#EVAL contractEnd's obligations          -- the responsibility
     ledger
#EVAL (flipBurden abuse)'s obligations   -- burdens flipped to the
     opponent
```

Figure 9: The lease judgement of [49] as executable L4. The `#ASSERT`s encode the reported outcome; the `#EVAL`s print the responsibility ledger that the `Writer` dimension accumulates.

Table 2: Burden of proof as the per-proposition choice of monoidal identity.

Monoid on `Bool`	identity	legal reading
$(\vee, \mathit{False})$	*False*	presumption of innocence
$(\wedge, \mathit{True})$	*True*	rebuttable presumption of the claim

(evidence can move off it); a *conclusive* presumption or a *jus cogens* norm is the absorbing element (no evidence moves it). A legal regime is thus a pair (*identity*, *threshold*)—who gets the benefit of the doubt, and how much doubt is tolerated; criminal law fixes (*innocent*, *beyond-reasonable-doubt*), encoding Blackstone's ratio [7] as, in effect, a decision-theoretic loss function over the asymmetric cost of error [26]. The monad unit law reads jurisprudentially: the presumption on its own moves nothing; only evidence ($\gg=$) does.

A.8 Correspondence and the answer-set image

The same non-monotonic content has three faces the bridge must keep aligned (Table 3). Relationalising L4 as in Section 6—functions to predicates by A-normal-form flattening with an added output argument—sends `Provable` and its negation-as-failure to answer-set default negation [16, 21]; stable-model enumeration becomes scenario search, and the deontic-temporal double binds of Section 5 reappear as unsatisfiable cores, the same loophole-as-exploit framing now extended to cover the burden layer. Encoding the lease factbase and evaluating `contract_end` with these combinators (Figure 9) reproduces the reported judgement: the landlord prevails, the approval-of-sublease defence fails for want of plausibility, the non-abuse defence is itself defeated by abuse of confidence, and the ledger attributes each fact to its bearer—establishing, incidentally, that *who bears a burden* (the plaintiff, for the constitutive facts) is independent of *who establishes it* (here, the defendant's admission).

Table 3: Three faces of the same defeasible content.

	PROLEG	**`Provable`**	**ASP**
defeater	`exception`	`notProven`	`not e`
closed world	implicit	`resolve`	default neg.
proof standard	`plausible`	the `Maybe`	weak constr.
burden bearer	`opposite`	`flipBurden`	tie-break
resp. map	—	`Writer`	literals/party

The upshot for the main thesis is that the constitutive/regulative split of Section 3 gains a third lane—evidentiality. The natural user-facing artefact is a *responsibility map* with three projections (who is on the hook, who decides, who must prove), each a (*position*, *party*) pair drawn from a different Hohfeldian register, and the value of having them as separate types is precisely that the transpiler cannot silently collapse one into another.

REFERENCES

[1] Layman E. Allen. 1957. Symbolic Logic: A Razor-Edged Tool for Drafting and Interpreting Legal Documents. *The Yale Law Journal* 66, 6 (1957), 833–879.

[2] Rajeev Alur and David L. Dill. 1994. A Theory of Timed Automata. *Theoretical Computer Science* 126, 2 (1994), 183–235.

[3] Jesper Andersen, Martin Elsman, Fritz Henglein, and Ken Friis Larsen. 2006. Compositional Specification of Commercial Contracts. *International Journal on Software Tools for Technology Transfer (STTT)* 8, 6 (2006), 485–516.

[4] Sergio Antoy and Michael Hanus. 2010. Functional Logic Programming. *Commun. ACM* 53, 4 (2010), 74–85.

[5] Gerd Behrmann, Alexandre David, and Kim G. Larsen. 2004. A Tutorial on UPPAAL. In *Formal Methods for the Design of Real-Time Systems (SFM-RT) (LNCS)*, Vol. 3185. Springer, 200–236.

[6] Trevor Bench-Capon, Michał Araszkiewicz, Kevin Ashley, et al. 2012. A History of AI and Law in 50 Papers: 25 Years of the International Conference on AI and Law. *Artificial Intelligence and Law* 20, 3 (2012), 215–319.

[7] William Blackstone. 1769. *Commentaries on the Laws of England, Book IV: Of Public Wrongs.* Clarendon Press, Oxford.

[8] Johan Bos. 2008. Wide-Coverage Semantic Analysis with Boxer. In *Proceedings of the 2008 Conference on Semantics in Text Processing (STEP).* College Publications, 277–286.

[9] Roderick M. Chisholm. 1963. Contrary-to-Duty Imperatives and Deontic Logic. *Analysis* 24, 2 (1963), 33–36.

[10] Alessandro Cimatti, Edmund Clarke, Enrico Giunchiglia, Fausto Giunchiglia, Marco Pistore, Marco Roveri, Roberto Sebastiani, and Armando Tacchella. 2002. NuSMV 2: An OpenSource Tool for Symbolic Model Checking. In *Computer Aided Verification (CAV) (LNCS)*, Vol. 2404. Springer, 359–364.

[11] Keith L. Clark. 1978. Negation as Failure. In *Logic and Data Bases*, Hervé Gallaire and Jack Minker (Eds.). Plenum Press, 293–322.

[12] George Coode. 1843. On Legislative Expression; or, The Language of the Written Law. In *Reprinted in E. A. Driedger, The Composition of Legislation.* Originally London: W. Benning.

[13] Silvia Crafa, Cosimo Laneve, Giovanni Sartor, and Adele Veschetti. 2023. Pacta Sunt Servanda: Legal Contracts in Stipula. *Science of Computer Programming* 225 (2023).

[14] James R. Curran, Stephen Clark, and Johan Bos. 2007. Linguistically Motivated Large-Scale NLP with C&C and Boxer. In *Proceedings of the 45th Annual Meeting of the ACL, Demo Session*. Association for Computational Linguistics, 33–36.

[15] Digital Asset. 2019. DAML: A Smart Contract Language for Distributed Ledgers. https://www.digitalasset.com/developers.

[16] Phan Minh Dung. 1995. On the Acceptability of Arguments and Its Fundamental Role in Nonmonotonic Reasoning, Logic Programming and n-Person Games. *Artificial Intelligence* 77, 2 (1995), 321–357.

[17] European Parliament and Council. 2016. Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union L 119, 1–88. See Art. 22 and Recital 71.

[18] European Parliament and Council. 2024. Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act). Official Journal of the European Union L, 2024/1689.

[19] Edward A. Feigenbaum. 1984. Knowledge Engineering: The Applied Side of Artificial Intelligence. *Annals of the New York Academy of Sciences* 426 (1984), 91–107.

[20] Paul Brillant Feuto Njonko, Sylviane Cardey, Peter Greenfield, and Walid El Abed. 2014. RuleCNL: A Controlled Natural Language for Business Rule Specifications. In *Controlled Natural Language (CNL 2014) (LNCS)*, Vol. 8625. Springer, 66–77.

[21] Michael Gelfond and Vladimir Lifschitz. 1988. The Stable Model Semantics for Logic Programming. In *Proceedings of the Fifth International Conference and Symposium on Logic Programming (ICLP/SLP)*. MIT Press, 1070–1080.

[22] Guido Governatori. 2005. Representing Business Contracts in RuleML. *International Journal of Cooperative Information Systems* 14, 2–3, 181–216.

[23] Guido Governatori, Francesco Olivieri, Antonino Rotolo, and Simone Scannapieco. 2013. Computing Strong and Weak Permissions in Defeasible Logic. *Journal of Philosophical Logic* 42, 6 (2013), 799–829.

[24] Michael Hanus. 2013. Functional Logic Programming: From Theory to Curry. *Programming Logics: Essays in Memory of Harald Ganzinger, LNCS* 7797 (2013), 123–168.

[25] H. L. A. Hart. 1961. *The Concept of Law*. Oxford University Press.

[26] Bruce L. Hay and Kathryn E. Spier. 1997. Burdens of Proof in Civil Litigation: An Economic Perspective. *The Journal of Legal Studies* 26, 2 (1997), 413–431.

[27] Frederick Hayes-Roth, Donald A. Waterman, and Douglas B. Lenat (Eds.). 1983. *Building Expert Systems*. Addison-Wesley.

[28] C. A. R. Hoare. 1978. Communicating Sequential Processes. *Commun. ACM* 21, 8 (1978), 666–677.

[29] C. A. R. Hoare. 1985. *Communicating Sequential Processes*. Prentice-Hall, Englewood Cliffs, NJ.

[30] Wesley Newcomb Hohfeld. 1913. Some Fundamental Legal Conceptions as Applied in Judicial Reasoning. *The Yale Law Journal* 23, 1 (1913), 16–59.

[31] Gerard J. Holzmann. 1997. The Model Checker SPIN. *IEEE Transactions on Software Engineering* 23, 5 (1997), 279–295.

[32] Tom Hvitved. 2011. *Contract Formalisation and Modular Implementation of Domain-Specific Languages*. Ph.D. Dissertation. Department of Computer Science, University of Copenhagen (DIKU).

[33] Tom Hvitved, Felix Klaedtke, and Eugen Zălinescu. 2012. A Trace-Based Model for Multiparty Contracts. *The Journal of Logic and Algebraic Programming* 81, 2 (2012), 72–98.

[34] Robert Kowalski and Akber Datoo. 2023. Logical English for Law and Education. In *Prolog: The Next 50 Years, LNCS*, Vol. 13900. Springer, 287–299.

[35] Robert Kowalski and Marek Sergot. 1986. A Logic-Based Calculus of Events. *New Generation Computing* 4, 1 (1986), 67–95.

[36] Tobias Kuhn. 2014. A Survey and Classification of Controlled Natural Languages. *Computational Linguistics* 40, 1 (2014), 121–170.

[37] Leslie Lamport. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

[38] Peter J. Landin. 1966. The Next 700 Programming Languages. *Commun. ACM* 9, 3 (1966), 157–166.

[39] Carl Malamud. 2020. Public.Resource.Org and Law.Gov: Making Primary Legal Materials Freely Available. https://public.resource.org/; see also *Georgia v. Public.Resource.Org, Inc.*, 590 U.S. 255 (2020).

[40] Denis Mérigoux, Nicolas Chataing, and Jonathan Protzenko. 2021. Catala: A Programming Language for the Law. In *Proceedings of the ACM on Programming Languages (ICFP)*, Vol. 5. ACM, 1–29.

[41] James Mohun and Alex Roberts. 2020. Cracking the Code: Rulemaking for Humans and Machines. OECD Working Papers on Public Governance No. 42, OECD Publishing, Paris.

[42] Object Management Group. 2019. Semantics of Business Vocabulary and Business Rules (SBVR), Version 1.5. OMG Document formal/19-10-02.

[43] Object Management Group. 2021. Decision Model and Notation (DMN), Version 1.3. OMG Document formal/21-02-01.

[44] Simon Peyton Jones, Jean-Marc Eber, and Julian Seward. 2000. Composing Contracts: An Adventure in Financial Engineering (Functional Pearl). In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, 280–292.

[45] Henry Prakken and Giovanni Sartor. 2015. Law and Logic: A Review from an Argumentation Perspective. *Artificial Intelligence* 227 (2015), 214–245.

[46] Aarne Ranta. 2004. Grammatical Framework: A Type-Theoretical Grammar Formalism. *Journal of Functional Programming* 14, 2 (2004), 145–189.

[47] G. M. Reed and A. W. Roscoe. 1988. A Timed Model for Communicating Sequential Processes. *Theoretical Computer Science* 58, 1–3 (1988), 249–261.

[48] A. W. Roscoe. 1997. *The Theory and Practice of Concurrency*. Prentice-Hall.

[49] Ken Satoh, Kento Asai, Takamune Kogawa, Masahiro Kubota, Megumi Nakamura, Yoshiaki Nishigai, Kei Shirakawa, and Chiaki Takano. 2011. PROLEG: An Implementation of the Presupposed Ultimate Fact Theory of Japanese Civil Code by PROLOG Technology. In *New Frontiers in Artificial Intelligence (JSAI-isAI) (LNCS)*, Vol. 6797. Springer, 153–164.

[50] John R. Searle. 1969. *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press.

[51] M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, and H. T. Cory. 1986. The British Nationality Act as a Logic Program. *Commun. ACM* 29, 5 (1986), 370–386.

[52] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. 1996. The Execution Algorithm of Mercury, an Efficient Purely Declarative Logic Programming Language. *The Journal of Logic Programming* 29, 1–3 (1996), 17–64.

[53] Tarmo Uustalu and Varmo Vene. 2008. Comonadic Notions of Computation. *Electronic Notes in Theoretical Computer Science* 203, 5 (2008), 263–284.

[54] Georg Henrik von Wright. 1951. Deontic Logic. *Mind* 60, 237 (1951), 1–15.

[55] Sandra Wachter, Brent Mittelstadt, and Luciano Floridi. 2017. Why a Right to Explanation of Automated Decision-Making Does Not Exist in the General Data Protection Regulation. *International Data Privacy Law* 7, 2 (2017), 76–99.

[56] Philip Wadler. 1992. The Essence of Functional Programming. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 1–14.